

Projet individuel — Algèbre Linéaire Numérique

Appliquée à l'Analyse de Données

Compte rendu

Hazem Douzi

31 mai 2026

Résumé

Ce rapport présente la résolution des trois exercices du projet, accompagnés des résultats numériques obtenus dans le notebook `hazem_douzi.ipynb`, des visualisations produites, de leur interprétation et d'une conclusion. On y traite : (1) la résolution de systèmes linéaires $Ax = b$ par méthodes directes (LU, Cholesky) et itératives (Jacobi, Gauss–Seidel) ; (2) les décompositions SVD et QR ; (3) les applications à l'analyse de données (ACP et compression d'image). L'exercice 4 rassemble les questions de synthèse. Tous les calculs utilisent une graine aléatoire fixe (`np.random.seed`), les résultats sont donc **reproductibles**.

Table des matières

1	Introduction	1
2	Exercice 1 — Résolution de systèmes linéaires	1
2.1	Méthode directe : décomposition LU	1
2.1.1	Construction d'une matrice SDP	1
2.1.2	Factorisation LU avec <code>scipy.linalg.lu</code>	2
2.1.3	Résolution par substitutions triangulaires	2
2.1.4	Vérification avec <code>numpy.linalg.solve</code>	2
2.2	Méthode de Cholesky	3
2.3	Méthodes itératives : Jacobi et Gauss–Seidel	3
3	Exercice 2 — Décompositions SVD et QR	4
3.1	Décomposition en valeurs singulières (SVD)	4
3.1.1	Matrice de données	4
3.1.2	Calcul et reconstruction	4
3.1.3	Approximation de rang 2 (Eckart–Young)	5
3.2	Décomposition QR	5
4	Exercice 3 — Application à l'analyse de données	5
4.1	Analyse en Composantes Principales (ACP)	5
4.1.1	Centrage	6
4.1.2	SVD des données centrées et variance expliquée	6
4.1.3	Projection sur le plan (CP1, CP2)	6
4.2	Compression d'image par SVD	7

1 Introduction

L'objectif du projet est de mettre en pratique les outils d'algèbre linéaire numérique de `numpy` et `scipy` et de les appliquer à l'analyse de données. Pour chaque question, on rappelle l'objectif, on présente les résultats du notebook, puis on les *interprète*. Les bibliothèques utilisées sont `numpy`, `scipy` et `matplotlib`.

2 Exercice 1 — Résolution de systèmes linéaires

On résout $Ax = b$ avec A une matrice 4×4 **symétrique définie positive (SDP)**. On compare des méthodes *directes* (LU, Cholesky), exactes à l'arrondi près, et des méthodes *itératives* (Jacobi, Gauss-Seidel), qui approchent la solution.

2.1 Méthode directe : décomposition LU

2.1.1 Construction d'une matrice SDP

Une matrice est SDP si $A = A^\top$ et $x^\top Ax > 0$ pour tout $x \neq 0$, ce qui équivaut à **toutes ses valeurs propres strictement positives**. On construit $A = B^\top B + 4I$ (avec B aléatoire) : $B^\top B$ est toujours symétrique semi-définie positive, et le terme $+4I$ rend toutes les valeurs propres strictement positives.

$$A = \begin{bmatrix} 4.581 & -0.731 & -0.248 & 0.659 \\ -0.731 & 8.029 & 2.590 & 0.433 \\ -0.248 & 2.590 & 10.104 & 3.384 \\ 0.659 & 0.433 & 3.384 & 7.442 \end{bmatrix}, \quad b = \begin{bmatrix} -1.013 \\ 0.314 \\ -0.908 \\ -1.412 \end{bmatrix}.$$

Table 1 – Valeurs propres de A et test de symétrie.

Valeurs propres	13.484	7.708	4.117	4.846
Toutes > 0 ?	Oui			$A = A^\top$: Vrai

Interprétation. Les quatre valeurs propres sont strictement positives et A est symétrique : A est bien **SDP**. Cela légitime l'emploi de Cholesky (§2.2) en complément de LU.

2.1.2 Factorisation LU avec `scipy.linalg.lu`

`scipy.linalg.lu(A)` renvoie P (permutation), L (triangulaire inférieure à diagonale unité) et U (triangulaire supérieure), avec la convention $A = PLU$, donc $P^\top A = LU$.

Table 2 – Matrices L et U obtenues ($P = I$: aucun échange de lignes nécessaire).

L				U			
1	0	0	0	4.581	-0.731	-0.248	0.659
-0.160	1	0	0	0	7.912	2.550	0.538
-0.054	0.322	1	0	0	0	9.268	3.246
0.144	0.068	0.350	1	0	0	0	6.173

Vérification : $P^\top A = LU \Rightarrow$ **Vrai**.

2.1.3 Résolution par substitutions triangulaires

Une fois $A = PLU$, résoudre $Ax = b$ revient à deux systèmes triangulaires peu coûteux :

$$\tilde{b} = P^\top b, \quad Ly = \tilde{b} \text{ (substitution avant)}, \quad Ux = y \text{ (substitution arrière)}.$$

On obtient

$$x \approx (-0.195056, 0.047759, -0.056848, -0.149426).$$

2.1.4 Vérification avec `numpy.linalg.solve`

La routine `numpy.linalg.solve` (LAPACK) sert de référence. Le test `np.allclose` renvoie **Vrai** : notre résolution coïncide avec la référence à la précision machine.

2.2 Méthode de Cholesky

Comme A est SDP, on peut écrire $A = LL^\top$. La résolution via `cho_factor` / `cho_solve` donne la **même** solution que LU. On compare ensuite temps et précision (résidu $\|Ax - b\|_2$, moyenné sur 1000 répétitions).

Table 3 – Comparaison LU vs Cholesky (temps indicatif, dépend de la machine).

	LU	Cholesky
Temps moyen (μs)	≈ 35	≈ 10
Résidu $\ Ax - b\ _2$	2.29×10^{-16}	4.00×10^{-16}

Interprétation. Précision : les deux résidus sont de l'ordre de 10^{-16} , soit la précision machine ($\varepsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$) ; l'écart est négligeable. **Temps** : Cholesky est ici ~ 2 – 3 fois plus rapide que LU car il exploite la symétrie et évite le pivotage. **Règle pratique** : dès que A est SDP, on préfère Cholesky.

2.3 Méthodes itératives : Jacobi et Gauss–Seidel

On décompose $A = D + (L + U)$ (D diagonale, L/U parties triangulaires strictes).

- **Jacobi** (mise à jour *simultanée*) : $x^{(k+1)} = D^{-1}(b - (L + U)x^{(k)})$.
- **Gauss–Seidel** (mise à jour *séquentielle*) : pour calculer $x_i^{(k+1)}$, on réutilise immédiatement les composantes déjà recalculées $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$.

Critère d'arrêt : $\|x^{(k+1)} - x^{(k)}\|_2 < \text{tol} = 10^{-6}$ (au plus 100 itérations).

Table 4 – Résultats des méthodes itératives (convergence vers la solution de LU).

	Jacobi	Gauss–Seidel
Nombre d'itérations	19	10
Résidu final $\ Ax - b\ _2$	3.90×10^{-6}	1.56×10^{-7}
Solution	identique à LU	

Table 5 – Norme de la correction $\|x^{(k+1)} - x^{(k)}\|_2$ à quelques itérations.

Itération	1	5	8	10	19
Jacobi	0.3074	0.0100	0.00135	0.000354	9×10^{-7}
Gauss–Seidel	0.2740	0.000814	6×10^{-6}	$\leq 10^{-6}$ (convergé)	—

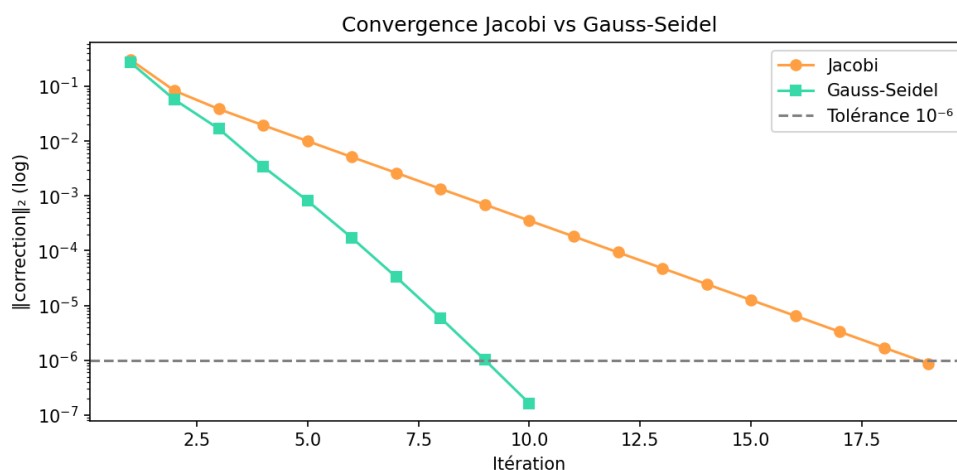


Figure 1 – Convergence comparée de Jacobi et Gauss–Seidel (échelle logarithmique). Gauss–Seidel atteint la tolérance 10^{-6} en deux fois moins d'itérations.

Interprétation. Gauss–Seidel converge en **10** itérations contre **19** pour Jacobi ($\sim 2 \times$ moins), avec un résidu final plus faible. Sa pente de convergence est plus raide car il exploite l'information « fraîche » à chaque pas. La convergence des deux méthodes est ici garantie : A est SDP et à diagonale dominante.

3 Exercice 2 — Décompositions SVD et QR

3.1 Décomposition en valeurs singulières (SVD)

3.1.1 Matrice de données

On simule les notes de 10 étudiants sur 5 matières (entiers de 0 à 19) :

$$D = \begin{bmatrix} 12 & 15 & 0 & 3 & 3 \\ 7 & 9 & 19 & 18 & 4 \\ 6 & 12 & 1 & 6 & 7 \\ 14 & 17 & 5 & 13 & 8 \\ 9 & 19 & 16 & 19 & 5 \\ 15 & 15 & 0 & 18 & 3 \\ 17 & 19 & 19 & 19 & 14 \\ 7 & 0 & 1 & 9 & 0 \\ 10 & 3 & 11 & 18 & 2 \\ 0 & 0 & 4 & 5 & 6 \end{bmatrix}.$$

3.1.2 Calcul et reconstruction

`numpy.linalg.svd` (option `full_matrices=False`) donne $D = U\Sigma V^\top$ avec $U \in \mathbb{R}^{10 \times 5}$, Σ de taille 5 et $V^\top \in \mathbb{R}^{5 \times 5}$.

Table 6 – Valeurs singulières de D et énergie associée.

σ_i	74.73	22.62	14.70	8.72	6.31
σ_1 concentre	$\frac{\sigma_1^2}{\sum_i \sigma_i^2} \approx \mathbf{86.9\%}$ de l'énergie.				

Reconstruction complète : $\|D - D_{\text{rec}}\|_F \approx 5.8 \times 10^{-14}$ (`np.allclose` $\Rightarrow$ **Vrai**) : la SVD reconstruit D **exactement**.

3.1.3 Approximation de rang 2 (Eckart–Young)

En ne gardant que les deux plus grandes valeurs singulières $D_2 = U_{:, :2} \text{diag}(\sigma_{1:2}) V_{2,:}^\top$, on obtient l'erreur relative de Frobenius :

$$\frac{\|D - D_2\|_F}{\|D\|_F} \approx \mathbf{22.7\%}.$$

Interprétation. D'après le théorème d'**Eckart–Young**, D_2 est la meilleure approximation de rang 2. Avec seulement **2 composantes sur 5**, on conserve $\approx 95\%$ de l'énergie ($1 - 0.227^2 \approx 0.95$) : excellent compromis taille/fidélité, qui motive la réduction de dimension de l'ACP.

3.2 Décomposition QR

`scipy.linalg.qr(D)` factorise $D = QR$ avec Q orthogonale (10×10) et R triangulaire supérieure (10×5). Pivots (diagonale de R) : $(-34.19, -15.65, 24.00, -11.78, 9.03)$.

Table 7 – Vérifications de la décomposition QR.

Propriété	Résultat
$Q^\top Q = I$ (colonnes orthonormées)	Vrai
$QR = D$	Vrai

Interprétation. Q est bien orthogonale et QR restitue exactement D . La QR est l'outil de référence pour les **moindres carrés** et la **stabilité numérique** (orthogonalisation des colonnes).

4 Exercice 3 — Application à l'analyse de données

4.1 Analyse en Composantes Principales (ACP)

On applique l'ACP à D : (1) centrage, (2) SVD de la matrice centrée, (3) projection sur les premières composantes principales.

4.1.1 Centrage

On soustrait la moyenne de chaque colonne. Moyennes par matière : (9.7, 10.9, 7.6, 12.8, 5.2) ; après centrage, chaque colonne est de moyenne nulle (à l'arrondi près). Le centrage est indispensable car l'ACP analyse la variance *autour de la moyenne*.

4.1.2 SVD des données centrées et variance expliquée

Les lignes de $V^\top$ sont les composantes principales (CP) ; σ_i^2 est proportionnel à la variance portée par la CP_i .

Table 8 – Variance expliquée par chaque composante principale.

Composante	CP1	CP2	CP3	CP4	CP5
σ_i (centré)	32.07	22.22	13.71	8.15	6.01
Variance (%)	56.7	27.2	10.4	3.7	2.0
Cumulée (%)	56.7	84.0	94.3	98.0	100

4.1.3 Projection sur le plan (CP1, CP2)

$Z = D_{\text{centré}} V_{:,2}$ donne les coordonnées des étudiants :

Table 9 – Coordonnées des 10 étudiants dans le plan CP1/CP2.

Étudiant	CP1	CP2	Étudiant	CP1	CP2
Ét.1	7.17	-10.27	Ét.6	-1.27	-7.52
Ét.2	-7.42	10.34	Ét.7	-18.07	-1.02
Ét.3	7.56	-5.10	Ét.8	13.25	4.11
Ét.4	-3.50	-7.47	Ét.9	-0.09	8.83
Ét.5	-11.97	1.32	Ét.10	14.34	6.76

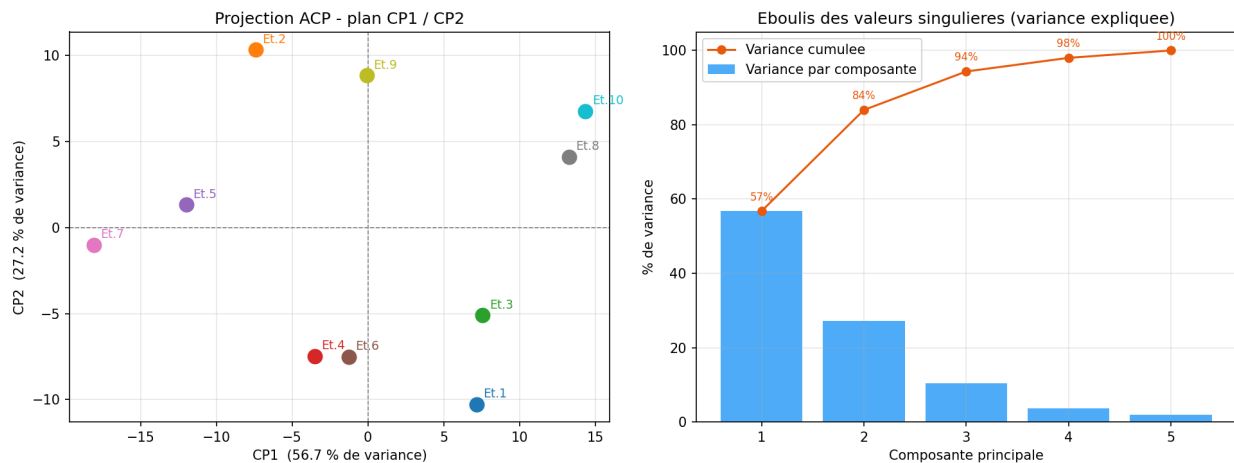


Figure 2 – À gauche : projection des étudiants dans le plan CP1/CP2. À droite : éboulis des valeurs singulières (variance expliquée par composante et cumulée).

Interprétation. CP1 ($\approx 56.7\%$) = **niveau général**. Tous les coefficients de l'axe sont de même signe : l'axe oppose les étudiants forts aux faibles. L'étudiant 7 (notes [17, 19, 19, 19, 14], le meilleur) se trouve à l'extrême gauche, l'étudiant 10 ([0, 0, 4, 5, 6], le plus faible) à l'extrême droite.

CP2 ($\approx 27.2\%$) = **contraste entre matières** (matières 1–2 contre matières 3–4) : à niveau global comparable, il sépare les profils « forts dans certaines matières / faibles dans d'autres » (ex. l'étudiant 2 [7, 9, 19, 18, 4] s'oppose à l'étudiant 1 [12, 15, 0, 3, 3]).

L'éboulis confirme que **2 composantes** suffisent (84 % de variance cumulée) : la projection 2D est fidèle.

4.2 Compression d'image par SVD

Une image en niveaux de gris est une matrice de pixels. En gardant les k premières valeurs singulières, on ne stocke que $k(m + n + 1)$ nombres au lieu de $m \times n$. On charge l'image (fichier local `image.png` sinon photo d'exemple de matplotlib, 600×512), on la convertit en niveaux de gris (pondération ITU-R BT.601), puis on applique la SVD.

Table 10 – Qualité vs taux de stockage selon le rang k (PSNR : plus élevé = meilleur). σ_1 capte déjà $\approx 73\%$ de l'énergie de l'image.

Rang k	Stockage (%)	$\ \text{erreur}\ _F$	PSNR (dB)
10	3.6	58.85	19.48
50	18.1	24.29	27.17
100	36.2	13.95	31.98
512 (complet)	185.5	≈ 0	∞



Figure 3 – Compression SVD : impact du rang sur la qualité visuelle (rangs 10, 50, 100 et image originale).

Interprétation. **rang 10** ($\sim 3.6\%$ de stockage) : seules les grandes structures subsistent, image floue (PSNR ~ 19.5 dB), bonne uniquement pour une vignette. **rang 50** ($\sim 18\%$) : image reconnaissable, PSNR ~ 27 dB. **rang 100** ($\sim 36\%$) : bonne qualité visuelle, PSNR ~ 32 dB, compression $\sim 3\times$. Un PSNR > 30 dB est généralement jugé « bonne qualité ». À noter : stocker la SVD *complète* (rang 512) coûte *plus* que l'image d'origine ($\sim 185\%$) ; tout l'intérêt est donc de **tronquer**.

5 Exercice 4 — Questions de synthèse

1. Performances (temps, précision) : méthodes directes vs itératives

Précision. Les méthodes directes (LU, Cholesky) atteignent la **précision machine** (résidu $\approx 2-4 \times 10^{-16} \approx \varepsilon_{\text{mach}}$) : elles résolvent le système exactement à l'arrondi flottant près. Les méthodes itératives s'arrêtent dès que la correction passe sous $\varepsilon = 10^{-6}$, d'où un résidu plus élevé (3.9×10^{-6} pour Jacobi, 1.6×10^{-7} pour Gauss-Seidel).

Temps. Sur notre matrice 4×4 dense, Cholesky est $\sim 2-3\times$ plus rapide que LU, et les itératives effectuent 10 à 19 itérations en $O(n^2)$ chacune. Mais pour de **grandes matrices creuses** ($n \sim 10^4-10^6$), les méthodes itératives deviennent indispensables : chaque itération coûte $O(\text{nnz})$ (nombre de non-zéros), alors que la factorisation LU dense coûte $O(n^3)$ en temps et $O(n^2)$ en mémoire.

Conclusion. Petites/moyennes matrices denses $\rightarrow$ **LU ou Cholesky** (précision machine garantie). Grandes matrices creuses ou systèmes issus d'EDP $\rightarrow$ **Jacobi, Gauss-Seidel**, ou des méthodes plus performantes (gradient conjugué, GMRES).

2. Impact de la réduction de dimension via SVD

Optimalité (Eckart–Young). Parmi toutes les matrices de rang $\leq k$, l’approximation $D_k = U_k \Sigma_k V_k^\top$ minimise l’erreur (normes de Frobenius et spectrale) : la SVD fournit la meilleure compression possible à rang fixé.

Effet concret sur D . $\sigma_1 \approx 74.7$ capte $\approx 87\%$ de l’énergie : les notes sont fortement corrélées entre matières (un « niveau général » explique l’essentiel). Garder 2 composantes conserve $\approx 95\%$ de l’énergie, suffisant pour la visualisation et l’analyse exploratoire.

Avantages : réduction du bruit (les petites σ capturent souvent du bruit), suppression de la multicolinéarité, lutte contre la « malédiction de la dimension », mise en évidence de structures latentes. **Limite :** l’erreur de rang 2 ($\approx 22.7\%$) signifie que certaines nuances individuelles sont perdues. Le choix du rang est un **compromis fidélité/complexité**.

3. Avantages et limites de l’ACP

Avantages. L’ACP est **non supervisée**, sans hyperparamètre critique, et produit des composantes **orthogonales** (décorrélées) qui éliminent la multicolinéarité. Elle maximise la variance conservée composante par composante (optimalité de la SVD), réduit le coût des algorithmes en aval (k-means, SVM, régression) et offre une **visualisation 2D/3D** naturelle. Sur D , les deux premières CP capturent $\approx 84\%$ de la variance.

Limites. L’ACP est **linéaire** : elle ne capte pas les structures non linéaires (spirales, clusters concentriques). Elle est **sensible à l’échelle** : sans standardisation, les matières à grande variance dominent. Les composantes ne sont pas toujours **interprétables** (CP1 est une combinaison de toutes les matières). Enfin, elle suppose que *variance = information utile*, ce qui est faux pour certaines tâches (les directions de faible variance peuvent être discriminantes).

Alternatives : **t-SNE / UMAP** (structures non linéaires, sans reconstruction), **Kernel PCA** (ACP dans un espace de caractéristiques via un noyau), **ICA** (sources indépendantes plutôt que simplement décorréliées).

6 Conclusion générale

Ce projet a permis de comparer concrètement les grandes familles de méthodes d’algèbre linéaire numérique. Pour la résolution de $Ax = b$, les méthodes **directes** (LU, Cholesky) offrent une précision machine et sont idéales pour les matrices denses de petite taille, Cholesky étant le choix privilégié quand A est SDP ; les méthodes **itératives** (Jacobi, Gauss–Seidel) prennent tout leur sens sur les grands systèmes creux, Gauss–Seidel convergeant environ deux fois plus vite que Jacobi. Les décompositions **SVD** et **QR** se sont révélées exactes et robustes : la SVD, optimale au sens d’Eckart–Young, est la clé de la réduction de dimension et de la compression (image compressée à qualité satisfaisante dès $\sim 36\%$ de stockage), tandis que l’**ACP** en a fourni une application directe en résumant fidèlement les profils des étudiants sur deux axes interprétables (niveau général et contraste entre matières). L’ensemble illustre le **compromis permanent entre coût, précision et fidélité** qui guide le choix d’une méthode numérique.